UNIVERSITY OF SCIENCE
FACULTY OF INFORMATION TECHNOLOGY

OBJECT-ORIENTED PROGRAMMING - CS202

SUPER MARIO BROS.

PROJECT REPORT

Class - Group:	Class 25A01 - Group 1
Students:	Nguyen Le Tran - 25125039 Le Quang Nguyen Phuc - 25125070 Le Quynh Anh - 25125071 Dinh Nguyen Hong Anh - 25125078
Instructors:	PhD Dinh Ba Tien MSc Truong Phuoc Loc MSc Ho Tuan Thanh

September 2026

Contents

Abstract and Introduction

Abstract

This report presents the architecture, implementation, and evaluation of a C++ recreation of Super Mario Bros. developed with SFML for CS202. The application combines a state-driven interface, tile maps, multiple characters, collision and physics, enemies and items, local multiplayer, sound, persistence, a map editor, stage progression, and a hidden stage reached through an animated pipe transition. The design emphasizes explicit ownership, interface-based collaboration, and patterns that solve concrete maintenance problems.

The implementation applies State, Mediator, Memento, Factory, Strategy, Command, and observer-style event propagation. The report identifies pattern participants in the code, explains design reasoning and trade-offs, and evaluates the integrated result. It also documents the player-to-pipe cutscene, scrolling cloud scenery, correction of a terrain-overdraw artifact, and the verification process.

1.1 Background

A real-time platform game is a demanding object-oriented case study because input, movement, animation, collision, content, audio, cameras, interfaces, and persistence must cooperate under frame-time constraints. If these responsibilities are concentrated in one class, small changes become risky. The project therefore uses states for screens, managers for runtime domains, and a mediator for cross-system policy.

1.2 Objectives and scope

- Build a complete playable C++/SFML game with a stable application loop.
- Demonstrate encapsulation, abstraction, inheritance, polymorphism, and RAII ownership.
- Support Mario/Luigi, one or two local players, three principal stages, and a hidden area.
- Keep the architecture extensible for new screens, blocks, entities, and levels.
- Integrate presentation features without replacing the established game interface.

Online multiplayer, network synchronization, a scripting engine, and a commercial content pipeline are outside the course scope.

1. Requirements and Project Scope

1.1 Functional requirements

Area	Required behavior	Implementation evidence
Application flow	Menu, selection, play, pause, win, game-over, leaderboard, editor	GameState hierarchy; StateManager
Player	Mario/Luigi movement, running, jump, shooting, gravity	PlayerManager; input commands
Collision	Ground, block, enemy, item, lift, hazard, flag	Managers and block strategies
Content	Three main levels and hidden area	LevelManager; TMX maps; warp data
Entities	Enemies, coins, mushroom, fire flower, star	EnemyFactory; entity managers
Interface	HUD, score, coins, lives, world, timer, menu feedback	HUDManager; UI states
Persistence	Save and resume relevant progress	GameMemento; SaveManager
Tools	Create and edit maps	MapEditorState
Presentation	Animated pipe transfer and safe cloud scenery	GameWorld transition/scenery

1.2 Non-functional requirements

Maintainability is supported by focused managers and narrow interfaces. Stability is supported by deferred state transitions, deterministic frame ordering, and RAII ownership. Responsiveness is supported by delta-time animation and texture preloading. The pipe cutscene suppresses normal input and unrelated simulation while a map hand-off is in progress.

1.3 Acceptance criteria

An acceptable integrated build must launch the complete interface, load available stages, support the chosen character/mode, resolve common collisions, create entities from map data, persist a valid snapshot, and complete both directions of the pipe cycle. Visual acceptance additionally requires a visible pipe-entry motion, an opaque hand-off, and cloud scenery that never covers terrain.

2. Team Composition and Integration

2.1 Team responsibilities

Member	Role	Responsibilities
Le Quynh Anh 25125071	Lead / Architecture / Integration	Architecture: GitHub, SFML, conventions, class design. Core flow: Game, StateManager, PlayState. Pattern: GameWorld as Mediator. Integration: Testing branch, pipe transition, scenery.
Nguyen Le Tran 25125039	Gameplay / Player / Collision	Player: movement, gravity, jump, controls. Collision: terrain, blocks, enemies, items. Characters: Mario, Luigi, local multiplayer. Support: player feedback and map editor.
Dinh Nguyen Hong Anh 25125078	Level / Entity / Factory / Sound	Levels: TMX loading and three main stages. Enemies: Goomba, Koopa, hazards, lifecycle. Items: coins and power-ups. Pattern/support: EnemyFactory and sound.
Le Quang Nguyen Phuc 25125070	UI / Persistence / Progression	Interface: HUD and main menu flow. States: pause, game-over, winning flow. Persistence: Memento save/load. Progression: locked card-based level selection.

2.2 Integration approach

Each member developed a bounded subsystem. All shared changes were reviewed and integrated into the Testing branch before becoming the common baseline. This protected the existing menu, gameplay, UI, maps, entities, persistence, and editor while new features were added.

Stage	Branch activity	Quality gate
1. Feature	Member-owned branch	Focused implementation and local build
2. Review	Compare with latest Testing	Inspect shared-file diffs and dependencies
3. Integrate	Merge into Testing	Resolve conflicts without replacing unrelated work
4. Validate	Testing becomes baseline	Build, launch, and run focused smoke checks

2.3 Conventions

Types use PascalCase; functions and locals use camelCase; members use the m_ prefix; interfaces begin with I. Smart pointers express lifetime, and comments preserve update-order and integration invariants.

3. System Architecture

3.1 Overall class and subsystem diagram

Game owns the application loop and delegates screen behavior to StateManager. PlayState is the gameplay composition root. GameWorld coordinates runtime managers through interfaces, while concrete managers own the actual entities and resources.

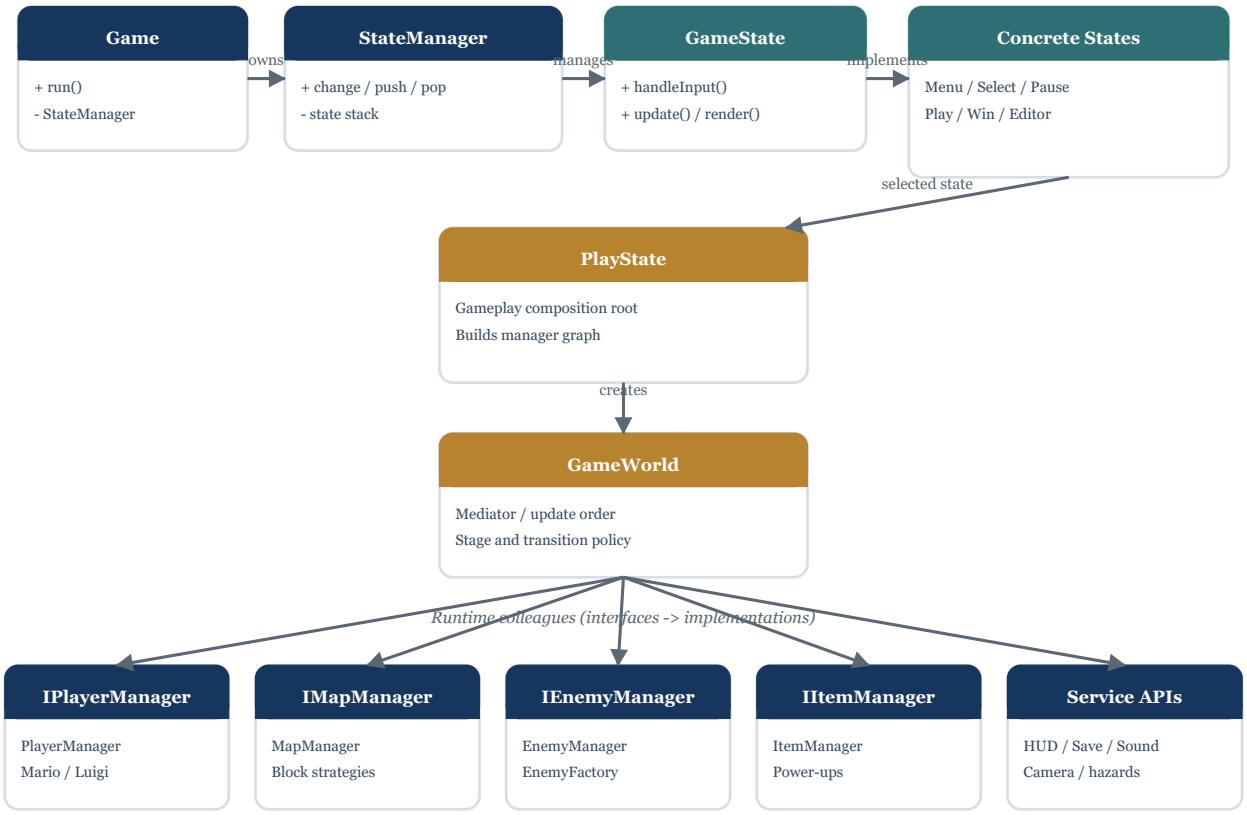

Figure 1. High-level ownership, state, mediator, interface, and implementation relationships.

The central design rule is that colleagues do not acquire unnecessary many-to-many ownership. Cross-system policy is routed through GameWorld; specialized algorithms remain inside their responsible managers.

3. System Architecture

3.2 Runtime flow

StateManager maintains a stack so PauseState can overlay PlayState. Requests are queued as Change, Push, or Pop and applied after the current callback returns. This prevents a state from deleting itself during input or update.

Flow	Purpose
Menu -> Character Select -> Mode Select	Collect session configuration
Mode Select -> Level Select -> Play	Choose an available stage and create gameplay
Play <-> Pause	Suspend/resume while preserving the world below
Play -> Intermission / Win	Controlled stage completion and progression
Play -> Game Over -> Initials / Leaderboard	Failure and score flow
Menu -> Map Editor	Separate content-authoring state

3.3 Frame update order

- Update map animation and cloud timing.
- Advance lifts before players so platform displacement is known.
- Update players and resolve terrain/lift movement.
- Consume any pending warp before unrelated systems update.
- Check flagpole, enemies, items, timer, hazards, death, and lives.
- Synchronize HUD and update the camera after final player positions.

3.4 Render order

The camera view draws map, scenery, lifts, hazards, enemies, items, and players. The default view then draws HUD elements and the full-screen transition fade. World-space score popups receive camera information while fixed labels remain screen-aligned.

4. Object-Oriented Design

4.1 Encapsulation

Managers hide entity collections, transition phases, timers, resources, and persistence details. External code cannot mutate pipe fields directly; GameWorld owns the complete phase invariant. SaveManager receives a snapshot instead of accessing every runtime object. Block behaviors receive IMapContext, a narrower mutation surface than concrete MapManager.

4.2 Abstraction and dependency inversion

IPlayerManager, IMapManager, IEnemyManager, IHUDManager, ISaveManager, IEnemyFactory, and IMapContext define capabilities needed by higher-level policy. GameWorld coordinates interfaces. EnemyManager accepts IEnemyFactory. Block strategies depend on IMapContext. These boundaries reduce includes, circular dependencies, and knowledge of constructors.

4.3 Inheritance and polymorphism

Concrete screens implement GameState. Enemy subclasses implement the Enemy contract. Mario and Luigi share the player mechanisms. Tile reactions implement IBlockBehavior. High-level loops therefore call common operations instead of containing repeated type-specific conditionals.

4.4 Composition and ownership

Game contains StateManager; PlayState owns GameWorld; GameWorld references collaborators; managers own live entities. EnemyFactory transfers exclusive enemy ownership as a unique pointer. RAII releases a session safely when its state is replaced.

4.5 SOLID assessment

- SRP: states, factories, managers, and persistence have focused roles.
- OCP: new states, blocks, enemies, and commands extend existing contracts.
- LSP: concrete implementations are used through base interfaces.
- ISP: specialized interfaces replace a universal manager API.
- DIP: high-level coordination depends on abstractions.

5. Design Patterns - State and Mediator

5.1 State Pattern

Problem. Screens have different input, update, render, and transition rules. A single conditional Game class would be difficult to extend.

Participants. GameState is the abstraction; MenuState, CharacterSelectState, ModeSelectState, LevelSelectState, PlayState, PauseState, IntermissionState, GameOverState, WinState, LeaderboardState, InitialsEntryState, and MapEditorState are concrete states; StateManager is the context.

Trade-off. The design creates more classes, but every screen has a clear responsibility. A state stack models overlays and deferred transitions prevent unsafe self-replacement.

5.2 Mediator Pattern

Problem. Player, map, enemies, items, HUD, camera, sound, hazards, and progression need ordered collaboration without circular ownership.

Participants. GameWorld is the Mediator. Manager interfaces are colleagues. PlayState constructs and registers the colleagues.

Mediator role	Project participant	Responsibility
Mediator	GameWorld	Orders cross-system gameplay and transitions
Composition root	PlayState	Creates and connects colleagues
Colleagues	Player, map, enemy, item managers	Own domain state and behavior
Services	HUD, save, sound, camera	Provide presentation and support functions

Trade-off. Coordination becomes visible and managers remain focused, but GameWorld must be kept at policy level to avoid becoming a God object.

5. Memento and Factory Patterns

5.3 Memento Pattern

Problem. Persistence requires stage, shared lives, per-player score and coins, configuration, and custom-map information without exposing every runtime field.

Participants. GameWorld is the Originator through createMemento and restoreFromMemento. GameMemento and PlayerStateMemento are snapshots. SaveManager is the Caretaker, reached through ISaveManager. PlayState coordinates load and resume.

Snapshot field	Reason
stage	Restores principal level progression
lives	Restores shared retry resource
players[] score and coins	Supports one or two players
GameConfig	Restores character and mode choices
customMapPath	Preserves editor/custom-map session

Trade-off. Runtime ownership and file persistence stay separate. Future schema changes should add explicit versioning.

5.4 Factory Pattern

Problem. TMX spawn data should not couple MapManager to concrete enemy constructors, textures, or callbacks.

Participants. IEnemyFactory defines preloadTextures and createEnemy. EnemyFactory maps EntitySpawnData to Enemy subclasses. EnemyManager accepts the abstraction and owns the returned objects.

Trade-off. New enemy registration still modifies the concrete factory, but constructor policy is localized and test factories can be injected.

5. Design Patterns - Strategy, Command, Observer

5.5 Strategy Pattern

IBlockBehavior defines solidity and directional contacts. Concrete behaviors implement pipes, question blocks, power-up blocks, hidden blocks, death zones, flagpoles, coins, bricks, and hazards. MapManager dispatches through the behavior interface; IMapContext limits side effects. The result replaces a broad collision switch with localized policies.

5.6 Command Pattern

Command declares execute(PlayerManager&). JumpCommand, StopJumpCommand, MoveLeftCommand, MoveRightCommand, StopHorizontalCommand, and ShootCommand are reusable actions. PlayerInputHandler is the invoker and translates configured SFML events or real-time key state. PlayerManager is the receiver. Input mapping can change without duplicating movement code, and pre-created commands avoid frame-time allocation.

5.7 Observer-style event propagation

Gameplay events must update career statistics and achievements without making player, enemy, or map classes responsible for persistence. PlayState registers callbacks on PlayerManager, EnemyManager, and MapManager. These callbacks obtain ISaveManager through GameWorld and call recordStat for coins, power-ups, stars, enemy kills, and broken blocks. GameWorld also records deaths, wins, co-op clears, and the Stage 1 speedrun condition.

SaveManager stores the counters and evaluates AchievementDefs. New unlocks are queued, drained by GameWorld, and displayed through IHUDManager. This is a simplified Observer implementation with one callback per event category rather than a reusable multi-subscriber event bus.

5.8 Pattern interaction

A PlayState (State) assembles GameWorld (Mediator). PipeBehavior (Strategy) queues a warp. Input arrives as Command objects. EnemyFactory creates map-defined enemies. GameWorld creates a GameMemento. Gameplay callbacks send career statistics to SaveManager, which evaluates achievements and returns unlock notifications to GameWorld.

6. Core Gameplay Implementation

6.1 Game loop and timing

The active state receives events, delta-time updates, and render calls. Delta time drives movement, animation, camera, scenery, and fades independently of frame rate. Menu states do not update gameplay resources; PlayState delegates runtime work to GameWorld.

6.2 Input and movement

PlayerInputHandler supports configurable bindings for one or two players. Discrete commands implement jump, stop-jump, and shoot; real-time commands implement horizontal intent. Run and Down are continuous conditions. Player update combines velocity, gravity, variable jump, collision correction, and animation state.

6.3 Collision responsibilities

Tile collision uses the map grid and block strategies. Enemy collision is handled after final player movement. Item collision uses the same final coordinates. Lifts update before players so platform displacement is correct. Fire bars and death zones are hazards. This order prevents one-frame errors in grounding, pickups, and animation.

6.4 Multiple characters and local multiplayer

The world can coordinate one or two player managers. Lives are shared, while score and coins are stored per player. In single-player the round ends when the player dies; in two-player it ends when both are dead. The memento uses a vector of player records rather than assuming a fixed count.

6.5 Completion and failure

Flagpole collision begins a controlled sequence that allows score and timer feedback before progression. Death reduces shared lives and either reloads the current level or marks the world as game over. The surrounding state system selects the appropriate intermission, win, or failure screen.

7. Level, Enemy, Item, and Map Systems

7.1 TMX pipeline

MapManager parses Tiled TMX tile layers into a TileType collision grid and object layers into typed EntitySpawnData. Data specifies what and where; code determines behavior.

TMX output	Runtime consumer	Responsibility
Tile grid	MapManager / IBlockBehavior	Collision and block effects
Enemy spawn data	EnemyFactory / EnemyManager	Concrete creation and lifecycle
Item spawn data	ItemManager	Collectables and power-ups
Lift data	LiftManager	Moving-platform behavior
Fire-bar data	FireBarManager	Rotating hazards
Warp data	MapManager / GameWorld	Hidden-map destination

7.2 Blocks

Directional strategy methods separate hitting from below, standing on top, and side contact. Question, hidden, brick, pipe, death-zone, and flagpole behavior is localized. IMapContext keeps behavior effects synchronized with map data.

7.3 Enemies and items

EnemyFactory preloads textures and returns exclusively owned enemies. EnemyManager updates, collides, and removes them. Items and block-spawned rewards update player state and feedback. The triggering player receives multiplayer block rewards, preventing coin ownership errors.

7.4 Progression

Three principal stages are selectable according to progression. Hidden rooms are warp destinations, not separate selectable stages, so entering and leaving them preserves the parent-stage context.

8. UI, Persistence, and Supporting Systems

8.1 HUD

HUDManager displays score, coins, lives, world, timer, character icon, toasts, and score popups. Its own update logic handles coin animation, warning colors, popup motion, and timer bonus. Fixed HUD elements use the default view, while world-space popups receive camera information.

8.2 Menus and progression interface

Character, mode, and level selection are concrete states. The level screen uses a card/grid layout so stages are visually distinct, and unopened entries remain locked. Pause is pushed over gameplay, preserving the world for resume. Game-over, initials, leaderboard, intermission, and win flows remain separate states.

8.3 Save/load sequence

- PlayState requests a snapshot from GameWorld.
- GameWorld captures stage, lives, player values, configuration, and custom map path.
- SaveManager persists the GameMemento through ISaveManager.
- On continue, PlayState loads configuration and initializes the required world objects.
- GameWorld restores the snapshot only after managers exist.

The schema intentionally stores durable progress rather than exact transient animation and enemy frames. Future versions should add a schema version and checkpoints if exact mid-level resume is required.

8.4 Sound and camera

Sound requests use an interface rather than exposing audio ownership. Camera tracking runs after final player positions. During pipe transfer, player and camera updates continue while unrelated systems are frozen, preserving a visible scripted cutscene.

9. Feature Case Study - Animated Pipe Transition

9.1 Design problem

An immediate map warp was logically correct but visually abrupt. The required experience was: visible movement into the pipe, fade to black, destination load, fade in, and restored control. It also had to support the side-facing hidden-room exit and optional second player without replacing the full game interface.

9.2 Trigger and phases

PipeBehavior queues a WarpRequest through IMapContext. The overworld entrance uses standing plus Down; the hidden-room exit uses side contact. GameWorld consumes the request after player movement, stores the destination, chooses a vertical or horizontal offset, and starts scripted travel.

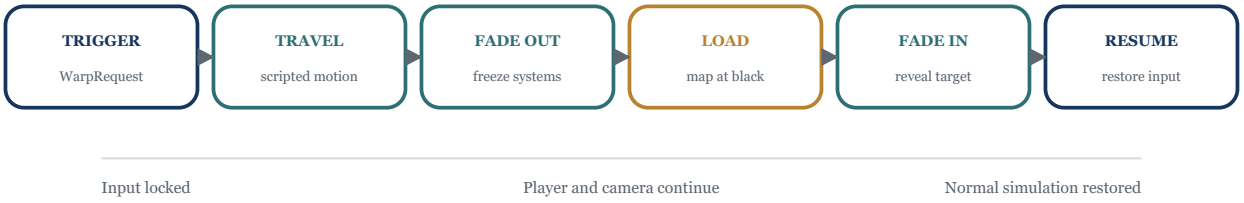

Figure 3. Explicit phase machine for the hidden-stage pipe transfer.

9.3 Safe hand-off

During any non-None phase, normal input and map/enemy/item/hazard updates are suspended. Players and camera still update. All active players must finish travel before FadeOut. The map loads only after the fade reaches black. FadeIn then reveals the destination; fields are cleared and input resumes only when it completes.

9.4 Evaluation

A scoped enumeration prevents invalid combinations of unrelated booleans. The transition belongs in the gameplay mediator because it coordinates multiple colleagues and a map hand-off. Future TMX properties could define direction, distance, and duration per pipe.

10. Feature Case Study - Background Scenery

10.1 Objective

The overworld required grass/bush detail and drifting clouds while preserving collision, camera behavior, and hidden-area identity. The final design keeps terrain decoration in the TMX map and adds only non-colliding clouds in a separate scenery pass.

10.2 Repetition and parallax

renderScenery calculates the visible camera interval, selects the first repeated sprite that could intersect the view, and draws until the right edge is passed. Two cloud crops use different base positions, spacing, height, and scroll factors. The scroll value advances with delta time and wraps with fmod to avoid unbounded growth. Hidden areas skip this pass.

10.3 Terrain-overdraw defect

An early repeated bush crop intermittently painted over the ground. The selected atlas rectangle shared pixels with a terrain row. At certain camera offsets, repeating that crop placed opaque terrain-colored pixels where the level designer had not intended them.

Layer	Final ownership	Reason
Ground / collision	TMX tile layer	Authoritative level geometry
Bushes and grass	TMX tile layer	Correct placement and atlas draw order
Drifting clouds	GameWorld scenery pass	Decorative parallax independent of collision
HUD	Default-view UI pass	Screen-aligned and camera independent

10.4 Correction and evaluation

Repeated bushes were removed from the parallax code; existing map bushes and grass remain. Only validated transparent cloud crops are repeated. The result preserves the requested environment, avoids terrain overdraw, and keeps level authorship in the correct layer.

11. Testing and Verification

11.1 Strategy

Verification combined build checks, direct execution, focused smoke tests, structural inspection, and visual review. Feature tests exercised full workflows rather than isolated endpoints, particularly the complete pipe-entry and exit cycle.

11.2 Functional matrix

ID	Scenario	Expected result	Status
T01	Launch integrated executable	Complete menu UI appears	Pass
T02	Select character/mode	Chosen configuration reaches levels	Pass
T03	Choose locked stage	Locked card cannot start	Pass
T04	Move/run/jump/release	Commands produce intended actions	Pass
T05	Ground and lift collision	Stable grounded state	Pass
T06	Hit question/hidden block	Correct behavior and reward	Pass
T07	Enemy and item interaction	Correct consequence and feedback	Pass
T08	Pause and resume	World preserved below overlay	Pass
T09	Save and continue	Snapshot restored	Pass
T10	Enter overworld pipe	Move down, fade, hidden map	Pass
T11	Exit hidden pipe	Move sideways and return	Pass
T12	Two-player pipe transfer	Wait for all active players	Pass
T13	Scroll overworld	Clouds repeat without terrain overdraw	Pass
T14	Reach flagpole	Controlled clear/win flow	Pass
T15	Lose shared lives	Game-over occurs once	Pass

11.3 Structural and visual evidence

Source inspection confirmed the reported pattern participants. Visual checkpoints covered the menu, pipe motion, full fade, hidden area, return path, and multiple camera offsets. Moving through repeated scenery positions was necessary because a single screenshot could not reveal an intermittent atlas-crop artifact.

12. Technical Challenges

12.1 Merge conflicts and preservation

GameWorld, PlayState, map data, and build configuration are integration hotspots. The team compared against the current integration branch, preserved unrelated changes, kept commits scoped, and inserted the new features into the established game instead of replacing UI or other members' work.

12.2 Safe state changes

A state can request replacement from inside its own callback. Immediate destruction would invalidate the call stack. StateManager queues Change, Push, and Pop operations and applies them afterward; fade phases delay visual replacement while retaining a valid state.

12.3 Frame-order dependencies

Lifts, players, enemies, items, hazards, and camera depend on each other's final positions. The mediator enforces a deterministic order. Warp consumption occurs immediately after player movement so unrelated systems do not advance on a map about to change.

12.4 Multiplayer cases

Shared lives, reward ownership, simultaneous pipe travel, and death conditions need explicit policy. The cutscene waits for all active players; two-player rounds end only when both are dead; block rewards remain associated with the triggering player; snapshots use a player vector.

12.5 Assets and coordinates

Sprite atlas crops must be pixel-accurate. The bush artifact was solved by returning terrain decoration to the tile layer. World and HUD coordinate systems are separated, and the transition fade uses the default view so it covers the entire window.

12.6 Lessons

- Integration safety is an architectural concern, not only a Git concern.
- Update and render order should be documented as invariants.
- Visual defects can require testing across camera motion, not only static frames.

13. Evaluation, Conclusion, and References

13.1 Strengths

The project provides a complete integrated flow: menus, level selection, gameplay, local multiplayer, content, UI, persistence, sound, editing, progression, and hidden-stage presentation. Pattern participants are identifiable in code. Interfaces and smart pointers reduce coupling and ownership risk. The pipe and scenery features enhance the existing application rather than bypassing it.

13.2 Limitations and future work

- Keep GameWorld at coordination level and extract specialized services as it grows.
- Formalize typed player/world events with subscribe and notify operations.
- Add save-schema versioning and checkpoint identifiers.
- Move pipe and scenery parameters into validated TMX properties.
- Add automated unit tests with injected factories and manager doubles.
- Support deterministic input replay for regression testing.

13.3 Conclusion

The delivered C++/SFML game demonstrates object-oriented design in a real-time application. State separates screens; Mediator orders cross-system work; Memento isolates persistence; Factory centralizes enemy creation; Strategy modularizes blocks; Command separates input from action; and observer-style callbacks separate gameplay events from career-stat and achievement processing. The architecture supports the course requirements and provides clear extension paths.

References

[1] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, Design Patterns: Elements of Reusable Object-Oriented Software, Addison-Wesley, 1994.

[2] SFML, SFML Documentation and Tutorials, https://www.sfml-dev.org/learn.php.

[3] Tiled, Tiled Map Editor Documentation, https://doc.mapeditor.org/.

[4] B. Stroustrup, The C++ Programming Language, 4th ed., Addison-Wesley, 2013.

[5] Project source evidence: src/core, src/states, src/world, src/entities, src/interfaces, and src/ui in the integrated testing codebase.